Contents

Resume-Writing Model Evaluation **1**
Results, Methodology, Prompts and Test Criteria . . . 1
Contents . . . 1
1. Executive summary . . . 1
2. What was tested, and what is not interchangeable . . . 2
3. Test A — free authoring . . . 2
4. Test B — full resume (faithfulness + structure) . . . 10
5. The year problem — why synthetic years must never reach output . . . 19
6. AA-PersonalityBench — a prior, not a score . . . 20
7. Corrections carried forward . . . 21
8. Limitations . . . 22
9. Reproducing . . . 22

Resume-Writing Model Evaluation

Results, Methodology, Prompts and Test Criteria

Generated: 2026-11-03
Harness: `resume_writing_v2/`
Total API spend for every experiment in this document: $3.03

This document is generated from the code, not written by hand. Every prompt, fixture and allowlist below is read directly from the module that ran, so it cannot drift from what was actually executed. Regenerate with `python resume_writing_v2/build_report.py`.

Contents

1. Executive summary
2. What was tested, and the three things that are NOT interchangeable
3. Test A — free authoring: inputs, prompt, scoring, results
4. Test B — full resume: inputs, prompt, scoring, results
5. The year problem — why fixture years must never reach output
6. AA-PersonalityBench: a prior, not a score
7. Corrections carried forward
8. Limitations
9. Reproducing

---

1. Executive summary

The decision this supports: which model should write resumes and STAR bullets for the Imaginator pipeline, and whether the quality gap justifies the price.

Headline results

- **48 model variants** measured in Test A (free authoring). **4 beat the incumbent significantly.**
  - **Qwen3.7 Plus** — 91.9 (Δ +7.8, t=3.50) at $0.00713/run, 2.90× the incumbent's cost
  - **MiniMax-M3** — 91.7 (Δ +7.7, t=3.50) at $0.00446/run, 1.81× the incumbent's cost
  - **Qwen3 Next 80B A3B Instruct** — 90.0 (Δ +6.0, t=2.89) at $0.00066/run, 0.27× the incumbent's cost

  - **Qwen3.6 35B A3B (Reasoning)** — 89.8 (Δ +5.8, t=2.62) at $0.00053/run, 0.21× the incumbent's cost
- **Best score-per-dollar:** `openai/gpt-oss-20b` — 85.3 at $0.00002/run.
- **Test B (full resume):** 35 of 39 variants passed every run on BOTH faithfulness and structure.

The three conclusions that matter

1. **Cheap can win.** `qwen/qwen3.6-35b-a3b` scores 89.8 against the incumbent's 84.1 (t=2.62) at about a fifth of the cost. Price and quality are not correlated in this set.
2. **Expensive can lose.** Grok 4.7 at `xhigh` costs 66× the incumbent per run and scores 72.0 — significantly worse. Claude Haiku 5.5 at `high` scores 73.8, also significantly worse.
3. **A pretty benchmark is not a fidelity benchmark.** An earlier bullets-only test ranked MiniMax M3.1 Flash first on format and last on faithfulness (24 invented items, 0/4 clean). The full-resume test now scores both and requires both.

---

2. What was tested, and what is not interchangeable

#	test	question	scored on
A	**Free authoring**	Can a model write a good resume from a verified profile?	gate-clamped 100-point rubric
B	**Full resume**	Can a model rewrite raw notes into a complete, correctly dated, correctly ordered resume without inventing anything?	faithfulness AND structure, both required
C	**AA-PersonalityBench**	What does a published writing benchmark say?	prior only — never a grade

A and B are different tasks. A gives the model a tidy profile and asks for prose. B gives it messy notes and an evidence boundary, then checks every date. A model can win one and lose the other — several did.

C is not a scoring axis. It was tested as a predictor and failed (§6).

Neither A nor B measures prose beauty. A fluent fabricator is worse than a plain faithful writer for this pipeline, so relative faithfulness is what is ranked.

---

3. Test A — free authoring

3.1 Input: the job description

```
Data Scientist (Data Science) - Boeing Defense, Space & Security

Position Responsibilities:
- Develop and deploy Generative AI and large language model solutions for
```

```
  defense analytics, including retrieval-augmented generation and fine-tuning.
- Build and maintain data pipelines and ETL processes in Python and SQL.
- Design evaluation frameworks for model performance, bias, and fairness in
  regulated environments.
- Work with distributed training and GPU infrastructure (H100 clusters).
- Containerize and orchestrate workloads with Docker and Kubernetes.
- Collaborate cross-functionally with engineering teams; support MLOps,
  monitoring, and CI/CD for production models.
- Ensure governance, compliance, and audit readiness for regulated defense data.

Required skills: Python, PyTorch, SQL, Docker, Kubernetes, AWS, LangChain.
```

3.2 Input: the verified candidate profile

This is the ONLY fact set a model may use. Every gate and every allowlist below is derived from it.

```
VERIFIED CANDIDATE FACTS — DO NOT INVENT ANYTHING OUTSIDE THIS PROFILE:

Name: Jeff Calderon
Location: Marysville, WA
Citizenship: U.S. Citizen
Contractor entity: Cogito Metric (C2C)

EDUCATION
- B.S. Data Analytics, Washington State University
- B.S. Physics, Washington State University
- A.A.S. Computer Science

CERTIFICATIONS
- CompTIA Security+ (Network Security+)
- Cybersecurity Analyst (ECC)
- Data Analytics certificate

EXPERIENCE (reverse chronological)
1. Cogito Metric (2024 - present) — built an NLP resume platform; 263 resumes
   processed; 500K in funding supported; 40% reduction in processing time;
   handles 30 requests per week; 13 production pipelines; 10+ TB of data.
2. NOAA — data engineering and analytics; weather and oceanographic datasets.
3. Naval Research Laboratory (NRL) — research computing support.
4. Capstone project — GPU cluster work, H100 exposure.

SKILLS
Python, PyTorch, TensorFlow, SQL, Docker, Kubernetes, AWS, LangChain, Pinecone,
Hadoop, Spark, Airflow.

DO NOT claim: a Master's degree, PhD, security clearance, or employment at
Google, Microsoft, Amazon, Meta, Apple, Netflix, OpenAI, or Anthropic.
```

3.3 The system prompt (verbatim)

```
You are an expert ATS-optimized resume writer for data science
and machine learning roles in the defense industry.
```

```
Rules:
1. Output ONLY the resume text. No preamble, no explanation, no markdown code
   fences, no commentary before or after.
2. Use ONLY facts present in the verified candidate profile. Invent nothing.
3. Plain text with clear section headings. Do NOT use markdown tables or
   multi-column layouts — ATS parsers cannot read them.
4. Bullets start with a strong action verb and include a quantified result
   where the profile provides one. No first-person pronouns in bullets.
5. Target 400-600 words.
```

3.4 The user message (verbatim)

Built by `task.build_prompt()`:

```
JOB DESCRIPTION:
Data Scientist (Data Science) - Boeing Defense, Space & Security

Position Responsibilities:
- Develop and deploy Generative AI and large language model solutions for
  defense analytics, including retrieval-augmented generation and fine-tuning.
- Build and maintain data pipelines and ETL processes in Python and SQL.
- Design evaluation frameworks for model performance, bias, and fairness in
  regulated environments.
- Work with distributed training and GPU infrastructure (H100 clusters).
- Containerize and orchestrate workloads with Docker and Kubernetes.
- Collaborate cross-functionally with engineering teams; support MLOps,
  monitoring, and CI/CD for production models.
- Ensure governance, compliance, and audit readiness for regulated defense data.

Required skills: Python, PyTorch, SQL, Docker, Kubernetes, AWS, LangChain.


VERIFIED CANDIDATE FACTS — DO NOT INVENT ANYTHING OUTSIDE THIS PROFILE:

Name: Jeff Calderon
Location: Marysville, WA
Citizenship: U.S. Citizen
Contractor entity: Cogito Metric (C2C)

EDUCATION
- B.S. Data Analytics, Washington State University
- B.S. Physics, Washington State University
- A.A.S. Computer Science

CERTIFICATIONS
- CompTIA Security+ (Network Security+)
- Cybersecurity Analyst (ECC)
- Data Analytics certificate

EXPERIENCE (reverse chronological)
1. Cogito Metric (2024 - present) — built an NLP resume platform; 263 resumes
   processed; 500K in funding supported; 40% reduction in processing time;
   handles 30 requests per week; 13 production pipelines; 10+ TB of data.
2. NOAA — data engineering and analytics; weather and oceanographic datasets.
3. Naval Research Laboratory (NRL) — research computing support.
```

```
4. Capstone project — GPU cluster work, H100 exposure.

SKILLS
Python, PyTorch, TensorFlow, SQL, Docker, Kubernetes, AWS, LangChain, Pinecone,
Hadoop, Spark, Airflow.

DO NOT claim: a Master's degree, PhD, security clearance, or employment at
Google, Microsoft, Amazon, Meta, Apple, Netflix, OpenAI, or Anthropic.


Write the tailored resume now.
```

3.5 Request parameters

parameter	value
`temperature`	0.2
`max_tokens`	16384 (reasoning headroom; a 4096 cap was consumed entirely by hidden reasoning in the first live run and returned an empty completion)
`usage.include`	true — so real billed cost is recorded per call
concurrency	4
retries	2
timeout	240s

3.6 Scoring: 100 points

Weights are asserted to sum to 100 at import. A predecessor harness declared 100, actually weighted 110, then clipped with `min(total, 100)` — which silently compressed every score.

category	points	what earns it
fidelity	30	no invented credentials; real employers and certs present
alignment	20	JD themes actually covered (7 themes)
bullets	12	action-verb ratio; average bullet length in band
metrics	10	verified numbers only
targeting	10	role named early; incumbent entity; prior employers
ats_structure	8	4 parseable sections
skills	5	required-stack coverage
brevity	5	400–600 word target band
total	**100**	`assert sum(WEIGHTS.values()) == 100`

Target word band for bullets: 7–28 words; resume: 320–700 (target 400–600).

3.7 Gates and the clamp

A gate failure caps the score, so a high score and a failed gate cannot coexist. This is the fix for a predecessor harness where 13 of 27 runs scored ≥90 while failing a gate.

gate	severity	checks
G1	**critical**	no invented degrees / employers / clearance
G2	**critical**	no unverified degree claimed (omission is not fatal)
G3	**critical**	identity intact (name, entity, citizenship)
G4	**critical**	output is the resume only — no fences, no preamble
G5–G10	non-critical	sections present, word count, first-person, contractor signal, role relevance, markdown table structure

Critical failure caps at **69**. Non-critical failure caps at **89**.

3.8 The allowlists (read from `gates.py`)

Verified employers: `noaa`, `naval research laboratory`, `nrl`, `washington state`, `cogito metric`, `capstone`

Verified degrees: `b.s. data analytics`, `b.s. physics`, `a.a.s. computer science`, `bachelor`

Verified certifications: `network security+`, `security+`, `comptia`, `cybersecurity analyst`, `ecc`, `data analytics`

Verified metrics: `263`, `500k`, `40%`, `10+ tb`, `13`

Forbidden degrees: `master`, `phd`, `ph.d`, `doctorate`, `m.s.`, `mba`, `m.eng`, `m.sc`

Forbidden employers: `google`, `microsoft`, `amazon`, `facebook`, `meta`, `apple`, `netflix`, `openai`, `anthropic`

Forbidden clearance claims: `top secret`, `ts/sci`, `secret clearance`, `ts sci`, `sci eligible`

> **Every VERIFIED_* entry must appear verbatim in the candidate profile.** `1000`, `4.0` and `2016` used to sit in the metrics list and appear nowhere in the profile — a model could invent that graduation year and be *credited* for it. `test_scorers.py` now enforces the invariant. See §7.

3.9 Significance method

Means alone are not published as a ranking. The test is Welch's t on per-run scores against the incumbent, with Cohen's d for effect size.

rule	value
significant	$\|t\| \geq 2.5$ **and** $\|d\| \geq 0.8$
otherwise	reported as `inside noise`
fewer than 3 scored runs	`low_n` — not a result

The p-value uses a normal approximation, which is stated rather than implied. At n=4–11 the conservative reading is that anything under $|t| \approx 2.5$ is not evidence.

3.10 Model set and rounds

round	set	runs/model	effort	file
1	every OpenRouter model with output ≤ $2/M	3	provider default	`results/contest_cheap_r1.js`
2	round-1 contenders within 4 points of control	8	provider default	`results/contest_cheap_r2.js`
3	named high/low-cost set	4	explicit	`results/contest_cheap_r3.js`

A variant is not a model. The unit is `(model_id, reasoning_effort)`. The same family at `xhigh` and at provider default is a different model and is never pooled.

Efforts are verified, not assumed. If an explicit `reasoning_effort` returns zero reasoning tokens, the row is flagged `effort_honoured: false` rather than reported under the requested effort. DeepSeek V4.1 Flash ignored `xhigh`.

Cost is measured, not modelled. `$/run` is the mean observed `usage.cost`, not price × an assumed token count.

3.11 Results — all measured variants

#	variant	n	mean	sd	Δ vs control	t	$/run	verdict
1	Qwen3.7 Plus	11	91.9	3.3	+7.8	3.50	$0.00713	significant
2	MiniMax-M3	14	91.7	3.3	+7.7	3.50	$0.00446	significant
3	MiMo-V2.6-Pro	4	90.5	3.6	+6.4	2.37	$0.00056	inside noise
4	Qwen3 Next 80B A3B Instruct	11	90.0	1.5	+6.0	2.89	$0.00066	significant
5	Qwen3.6 35B A3B (Reasoning)	11	89.8	3.0	+5.8	2.62	$0.00053	significant
6	Inkling Small	11	89.7	4.1	+5.6	2.38	$0.00647	inside noise
7	Inkling (xhigh) · xhigh	4	88.6	3.6	+4.5	1.67	$0.01818	inside noise
8	Qwen3 Coder Next	11	88.4	4.6	+4.4	1.79	$0.00045	inside noise

#	variant	n	mean	sd	Δ vs control	t	$/run	verdict
9	Gemini 3.8 Flash (high) · high	4	88.0	3.3	+3.9	1.51	$0.01302	inside noise
10	Nemotron 3 Ultra 550B	4	87.5	12.9	+3.5	0.52	$0.00406	inside noise
11	Mistral Large 4 Preview	4	87.3	5.1	+3.3	1.01	$0.01793	inside noise
12	DeepSeek V4.1 Flash (max) · xhigh	4	86.8	3.4	+2.7	1.05	$0.00028	inside noise
13	Qwen3.5 9B (Reasoning)	11	86.4	5.8	+2.3	0.88	$0.00003	inside noise
14	GLM-5.3-Flash	15	86.2	6.0	+2.2	0.85	$0.00061	inside noise
15	DeepSeek V4.1 Flash 1017 (max) · xhigh	4	85.6	6.3	+1.5	0.41	$0.00027	inside noise
16	gpt-oss-20b (High)	11	85.3	3.4	+1.2	0.54	$0.00002	inside noise
17	DeepSeek V4.1 Flash (incumbent) · default	11	84.1	6.7	+0.0	—	$0.00246	control
18	GPT-3.5 Turbo	4	84.1	1.4	+0.1	0.03	$0.00198	inside noise
19	MiniMax-M3.1-Flash	4	83.3	0.9	-0.8	-0.37	$0.00036	inside noise
20	gpt-oss-120b (High)	3	82.9	3.2	-1.1	-0.41	$0.00021	inside noise
21	Qwen3 Omni 30B A3B (Reasoning)	3	82.8	3.9	-1.3	-0.43	$0.00064	inside noise

#	variant	n	mean	sd	Δ vs control	t	$/run	verdict
22	Nemotron 3.5 Super VL	4	82.7	1.4	-1.4	-0.66	$0.00030	inside noise
23	LongCat 2.0	3	81.6	6.6	-2.5	-0.57	$0.01076	inside noise
24	Hy3	3	79.9	6.5	-4.1	-0.97	$0.00349	inside noise
25	Solar Pro 4	3	79.1	8.8	-5.0	-0.91	$0.00039	inside noise
26	DeepSeek V4 Flash Vision (Max)	2	78.8	5.4	—	—	$0.00511	low_n
27	Qwen3.8 27B (xhigh) · xhigh	3	78.8	4.8	-5.3	-1.54	$0.01546	inside noise
28	Qwen3.8 27B (Xhigh) · default	2	77.7	2.1	—	—	$0.00297	low_n
29	Gemma 4 31B (Non-reasoning)	3	76.7	4.6	-7.3	-2.20	$0.00020	inside noise
30	GPT-5.4 mini (non-reasoning)	4	76.3	2.2	-7.8	-3.41	$0.00244	significant
31	Beam 1.0	4	75.6	1.9	-8.4	-3.79	$0.00568	significant
32	NVIDIA Nemotron 3 Nano 30B A3B (Reason-ing)	6	74.4	8.4	-9.7	-2.43	$0.00060	inside noise
33	Llama 4 Maver-ick	7	74.4	3.0	-9.7	-4.21	$0.00026	significant
34	GPT-6.1 Sol (max) · xhigh	4	74.2	12.0	-9.9	-1.56	$0.06311	inside noise
35	Claude Haiku 5.5 (max) · high	4	73.8	3.4	-10.3	-3.89	$0.00159	significant

#	variant	n	mean	sd	Δ vs control	t	$/run	verdict
36	Qwen3.8 Max (0902)	4	73.8	1.6	-10.3	-4.79	$0.06538	significant
37	Kimi K3.1 Pro	4	73.5	0.9	-10.6	-5.17	$0.03803	significant
38	Llama 4 Scout	3	73.2	5.8	-10.8	-2.78	$0.00028	significant
39	Grok 4.7 (xhigh) · xhigh	4	72.0	3.3	-12.1	-4.66	$0.16239	significant
40	Grok 4.6 (xhigh) · xhigh	4	71.2	2.8	-12.8	-5.23	$0.03602	significant
41	GPT-6 Luna (max) · xhigh	4	71.2	2.4	-12.9	-5.52	$0.00394	significant
42	Llama 3.3 Instruct 70B	3	69.9	0.0	-14.2	-7.05	$0.00019	significant
43	Nemotron 3.5 Light-ning	1	69.0	—	—	—	$0.00060	low_n
44	Step 5 Preview	4	68.8	27.6	-15.2	-1.09	$0.03508	inside noise
45	Granite 4.2 8B	3	68.6	4.2	-15.4	-4.91	$0.00204	significant
46	Gemma 4 26B A4B (Reason-ing)	3	68.4	2.0	-15.6	-6.74	$0.00008	significant
47	Muse Glim-mer (high) · high	4	67.9	4.0	-16.1	-5.66	$0.00330	significant
48	Reka Flash 3	1	23.1	—	—	—	$0.00331	low_n

Control = DeepSeek V4.1 Flash at provider default.

---

4. Test B — full resume (faithfulness + structure)

4.1 Input: raw experience notes

Deliberately messy — this is what the pipeline actually receives.

```
[
  {
    "company": "Cogito Metric",
    "role": "Data Engineer",
    "duration": "04/2024 - Present",
    "bullets": [
      "Built the resume parsing pipeline that processes incoming candidate resumes",
      "Reduced how long the parsing step takes by about 40% after reworking it",
      "Worked with Postgres and Python for the storage layer",
      "Handled a fairly large amount of archived document data",
      "Talked to the recruiting team about what fields they needed"
    ]
  },
  {
    "company": "NOAA",
    "role": "Data Analyst",
    "duration": "06/2021 - 03/2024",
    "bullets": [
      "Made charts and reports for weather data on a weekly cadence",
      "Cleaned messy oceanographic datasets before analysis",
      "Presented findings to a small group of scientists"
    ]
  }
]
```

4.2 Input: education and certifications

> **⚠ THESE DATES ARE SYNTHETIC — DO NOT USE IN PRODUCTION**
>
> The school years and certificate dates below are **invented for this test**. They exist only so that "did the model copy the year exactly?" is measurable. They are **not** claims about the candidate and must never be copied into any prompt shown to a real user or into any generated resume. Full reasoning in §5.

```
[
  {
    "school": "Washington State University",
    "credential": "B.S. Data Analytics",
    "years": "2015 - 2019"
  },
  {
    "school": "Washington State University",
    "credential": "B.S. Physics",
    "years": "2013 - 2017"
  },
  {
    "school": "Bates Technical College",
    "credential": "A.A.S. Computer Science",
    "years": "2011 - 2013"
  }
]
```

```
[
  {
    "name": "CompTIA Security+",
    "issuer": "CompTIA",
```

```
    "date": "03/2023"
  },
  {
    "name": "Cybersecurity Analyst",
    "issuer": "ECC",
    "date": "07/2022"
  },
  {
    "name": "Data Analytics Certificate",
    "issuer": "Coursera",
    "date": "11/2021"
  }
]
```

4.3 Input: verified metrics (the only numbers allowed)

```
[
  {
    "value": "40",
    "unit": "%",
    "kind": "percent"
  },
  {
    "value": "30",
    "unit": "hours",
    "kind": "duration"
  }
]
```

4.4 Input: job description (context only, never evidence)

```
Data Scientist (Data Science) - Boeing Defense, Space & Security
- Develop and deploy Generative AI and large language model solutions,
  including retrieval-augmented generation and fine-tuning.
- Build data pipelines and ETL processes in Python and SQL.
- Work with distributed training and GPU infrastructure (H100 clusters).
- Containerize workloads with Docker and Kubernetes.
- Collaborate cross-functionally; support MLOps and CI/CD.
Required: Python, PyTorch, SQL, Docker, Kubernetes, AWS, LangChain.
```

4.5 The system prompt (verbatim)

```
You are an expert ATS-optimized resume writer producing a complete,
plain-text resume for a technical role.

### OUTPUT FORMAT (MANDATORY)
- Plain text ONLY. No markdown, no tables, no pipe characters, no code fences,
  no commentary before or after the resume.
- Use these three section headings, spelled exactly:
    PROFESSIONAL EXPERIENCE
    EDUCATION
    CERTIFICATIONS
```

```
### STRUCTURE (MANDATORY)
1. EXPERIENCE: every job from the input, in REVERSE-CHRONOLOGICAL order (most
   recent first). Do not reorder, merge, or drop a job.
2. DATES: copy the employment dates EXACTLY from the input Duration field,
   including month and year ("04/2024 - Present"). Do not reformat, round, or
   omit the month.
3. EDUCATION: one line per credential, copied exactly from the input, including
   the school and the year range exactly as given ("2015 - 2019"). Do NOT invent
   or adjust a graduation year.
4. CERTIFICATIONS: one line per certificate with the exact issue date from the
   input ("03/2023"). Do not invent a date.
5. Each job gets 2-4 bullets.

### TRUTH CONSTRAINTS (VIOLATION = FAILURE)
1. COMPANIES: use ONLY these companies, spelled exactly:
   ["Cogito Metric", "NOAA"]
2. JOB TITLES: keep the real role hierarchy; do not promote.
3. TECHNOLOGY - EVIDENCE ONLY: name only technologies present in the input.
   A technology that appears only in the job description is NOT evidence.
4. METRICS - EVIDENCE ONLY: use a number only when it is in the verified metrics
   list, and keep its unit attached. A bullet without a metric is not a failure.
5. No invented employers, schools, credentials, clearance, or awards.

### STYLE (MANDATORY)
- Each bullet 15-25 words, starting with a strong action verb ("Built",
  "Engineered", "Reduced", "Refactored"). Not "Worked on" or "Responsible for".
```

4.6 The user message (verbatim)

```
SOURCE OF TRUTH — the candidate's real history. Use NOTHING outside this.

EXPERIENCE:
[
  {
    "company": "Cogito Metric",
    "role": "Data Engineer",
    "duration": "04/2024 - Present",
    "bullets": [
      "Built the resume parsing pipeline that processes incoming candidate resumes",
      "Reduced how long the parsing step takes by about 40% after reworking it",
      "Worked with Postgres and Python for the storage layer",
      "Handled a fairly large amount of archived document data",
      "Talked to the recruiting team about what fields they needed"
    ]
  },
  {
    "company": "NOAA",
    "role": "Data Analyst",
    "duration": "06/2021 - 03/2024",
    "bullets": [
      "Made charts and reports for weather data on a weekly cadence",
      "Cleaned messy oceanographic datasets before analysis",
      "Presented findings to a small group of scientists"
    ]
```

```
  }
]

EDUCATION (copy these years exactly):
[
  {
    "school": "Washington State University",
    "credential": "B.S. Data Analytics",
    "years": "2015 - 2019"
  },
  {
    "school": "Washington State University",
    "credential": "B.S. Physics",
    "years": "2013 - 2017"
  },
  {
    "school": "Bates Technical College",
    "credential": "A.A.S. Computer Science",
    "years": "2011 - 2013"
  }
]

CERTIFICATIONS (copy these dates exactly):
[
  {
    "name": "CompTIA Security+",
    "issuer": "CompTIA",
    "date": "03/2023"
  },
  {
    "name": "Cybersecurity Analyst",
    "issuer": "ECC",
    "date": "07/2022"
  },
  {
    "name": "Data Analytics Certificate",
    "issuer": "Coursera",
    "date": "11/2021"
  }
]

VERIFIED METRICS (the ONLY numbers you may use):
- 40 % (percent)
- 30 hours (duration)

TARGET JOB DESCRIPTION — context only, never an evidence source:
Data Scientist (Data Science) - Boeing Defense, Space & Security
- Develop and deploy Generative AI and large language model solutions,
  including retrieval-augmented generation and fine-tuning.
- Build data pipelines and ETL processes in Python and SQL.
- Work with distributed training and GPU infrastructure (H100 clusters).
- Containerize workloads with Docker and Kubernetes.
- Collaborate cross-functionally; support MLOps and CI/CD.
Required: Python, PyTorch, SQL, Docker, Kubernetes, AWS, LangChain.
```

```
Write the complete resume now, with the three required section headings and
every date copied exactly from the source above.
```

4.7 Request parameters

parameter	value
`temperature`	0.3 — the production drafter's own setting
`max_tokens`	8192 (1200 for `gpt-3.5-turbo-0613`, whose context is 4095)
`reasoning_effort`	same value Test A used for that variant, so a row in both tables describes the same configuration
runs per variant	3

4.8 Test criteria — two contracts, both required

Faithfulness

check	rule
invented technology	any of 23 JD-only technologies in a bullet, word-boundary matched
invented company	an org-like token outside the allowed set
unverified number	a number not in the verified metrics list
unit stripped	a verified value whose unit was dropped (`30` instead of `30 hours`)

JD-only technologies (using any of these is fabrication): `airflow`, `aws`, `ci/cd`, `cuda`, `docker`, `fine-tun`, `finetun`, `genai`, `gpu`, `gpus`, `h100`, `hadoop`, `kubernetes`, `langchain`, `llm`, `llms`, `mlops`, `pinecone`, `pytorch`, `rag`, `retrieval-augmented`, `spark`, `tensorflow`

Allowed technologies: `csv`, `excel`, `git`, `jupyter`, `numpy`, `pandas`, `parquet`, `postgres`, `postgresql`, `python`, `sql`, `tableau`

Structure

check	rule
required sections	`PROFESSIONAL EXPERIENCE`, `EDUCATION`, `CERTIFICATIONS`
chronology	source companies appear in reverse-chronological order
employment dates	start **and** end copied exactly, month included
education years	both years of each range exact, and no invented year
certification dates	exact issue date per certificate
formatting	plain text; no markdown table, no code fence

Bullet format (reported separately, not part of `pass`): 15–25 words per bullet, strong action verb first, one thought per line.

Truncation is its own column. A run that uses ≥98% of its `max_tokens` budget was cut off mid-sentence; that is not a wrong date.

4.9 Word bands and verb lists (read from the module)

Strong verbs: `accelerated`, `analyzed`, `architected`, `automated`, `built`, `cleaned`, `collaborated`, `consolidated`, `created`, `cut`, `delivered`, `deployed`, `designed`, `developed`, `eliminated`, `engineered`, `implemented`, `improved`, `instrumented`, `led`, `migrated`, `optimized`, `partnered`, `presented`, `processed`, `reduced`, `refactored`, `reworked`, `scaled`, `standardized`, `streamlined`, `validated`

Vague verbs (penalised): `worked on`, `responsible for`, `strategized`, `pioneered`, `helped with`, `assisted with`, `was involved in`, `participated in`, `handled`

4.10 Results — pass rate per variant

#	variant	n	pass	faith	struct	truncated	15–25w band	$/run	Test A
1	Qwen3.7 Plus	3	**100%**	3/3	3/3	0/3	100%	$0.00592	91.9
2	MiniMax-M3	3	**100%**	3/3	3/3	0/3	100%	$0.00252	91.7
3	MiMo-V2.6-Pro	3	**100%**	3/3	3/3	0/3	100%	$0.00070	90.5
4	Qwen3 Next 80B A3B Instruct	3	**100%**	3/3	3/3	0/3	33%	$0.00041	90.0
5	Qwen3.6 35B A3B (Reasoning)	3	**100%**	3/3	3/3	0/3	100%	$0.00053	89.8
6	Inkling Small	3	**100%**	3/3	3/3	0/3	100%	$0.00533	89.7
7	Inkling (xhigh) · xhigh	3	**100%**	3/3	3/3	0/3	95%	$0.02413	88.6
8	Qwen3 Coder Next	3	**100%**	3/3	3/3	0/3	4%	$0.00032	88.4
9	Nemotron 3 Ultra 550B	3	**100%**	3/3	3/3	0/3	90%	$0.00665	87.5
10	DeepSeek V4.1 Flash (max) · xhigh	3	**100%**	3/3	3/3	0/3	89%	$0.00033	86.8
11	Qwen3.5 9B (Reasoning)	3	**100%**	3/3	3/3	0/3	7%	$0.00008	86.4
12	GLM-5.3-Flash	3	**100%**	3/3	3/3	0/3	29%	$0.00011	86.2

#	variant	n	pass	faith	struct	truncated	15–25w band	$/run	Test A
13	DeepSeek V4.1 Flash 1017 (max) · xhigh	3	**100%**	3/3	3/3	0/3	100%	$0.00035	85.6
14	gpt-oss-20b (High)	3	**100%**	3/3	3/3	0/3	0%	$0.00002	85.3
15	DeepSeek V4.1 Flash (incum-bent) · default	3	**100%**	3/3	3/3	0/3	90%	$0.00035	84.1
16	GPT-3.5 Turbo	3	**100%**	3/3	3/3	0/3	14%	$0.00154	84.1
17	MiniMax-M3.1-Flash	3	**100%**	3/3	3/3	0/3	90%	$0.00024	83.3
18	gpt-oss-120b (High)	3	**100%**	3/3	3/3	0/3	100%	$0.00021	82.9
19	Qwen3 Omni 30B A3B (Rea-soning)	3	**100%**	3/3	3/3	0/3	0%	$0.00121	82.8
20	Nemotron 3.5 Super VL	3	**100%**	3/3	3/3	0/3	0%	$0.00017	82.7
21	LongCat 2.0	3	**100%**	3/3	3/3	0/3	100%	$0.00396	81.6
22	Hy3	3	**100%**	3/3	3/3	0/3	100%	$0.00186	79.9
23	DeepSeek V4 Flash Vision (Max)	3	**100%**	3/3	3/3	0/3	100%	$0.00190	78.8
24	Qwen3.8 27B (xhigh) · xhigh	3	**100%**	3/3	3/3	0/3	100%	$0.00462	78.8

#	variant	n	pass	faith	struct	truncated	15–25w band	$/run	Test A
25	Qwen3.8 27B (Xhigh) · default	3	**100%**	3/3	3/3	0/3	100%	$0.00241	77.7
26	Gemma 4 31B (Non-reasoning)	3	**100%**	3/3	3/3	0/3	94%	$0.00021	76.7
27	GPT-5.4 mini (non-reasoning)	3	**100%**	3/3	3/3	0/3	75%	$0.00197	76.3
28	Beam 1.0	3	**100%**	3/3	3/3	0/3	100%	$0.00270	75.6
29	Llama 4 Maverick	3	**100%**	3/3	3/3	0/3	14%	$0.00034	74.4
30	GPT-6.1 Sol (max) · xhigh	3	**100%**	3/3	3/3	0/3	100%	$0.01769	74.2
31	Qwen3.8 Max (0902)	3	**100%**	3/3	3/3	0/3	100%	$0.02089	73.8
32	Llama 3.3 Instruct 70B	3	**100%**	3/3	3/3	0/3	0%	$0.00020	69.9
33	Granite 4.2 8B	3	**100%**	3/3	3/3	0/3	100%	$0.00117	68.6
34	Gemma 4 26B A4B (Reasoning)	3	**100%**	3/3	3/3	0/3	39%	$0.00007	68.4
35	Muse Glimmer (high) · high	3	**100%**	3/3	3/3	0/3	100%	$0.00391	67.9
36	Solar Pro 4	2	**50%**	2/2	1/2	0/2	0%	$0.00019	79.1
37	NVIDIA Nemotron 3 Nano 30B A3B (Reasoning)	2	**50%**	1/2	1/2	0/2	100%	$0.00130	74.4

#	variant	n	pass	faith	struct	truncated	15–25w band	$/run	Test A
38	Gemini 3.8 Flash (high) · high	3	**33%**	2/3	1/3	3/3	67%	$0.03125	88.0
39	Llama 4 Scout	3	**33%**	3/3	1/3	0/3	0%	$0.00026	73.2
40	Reka Flash 3	1	**0%**	0/1	0/1	1/1	37%	$0.00174	25.1

`pass` = faithfulness-clean AND structurally-clean on the same run.

4.11 What Test B showed

1. **Structure was rarely the failure.** Almost every variant copied dates and years exactly. The hard part was the **15–25 word band**, where several models scored 0–7%.
2. **One variant leaked its reasoning.** Gemini 3.8 Flash at `high` emitted internal word-count deliberation instead of a resume on all three runs, and hit the 8192-token cap each time.
3. **Test A does not predict Test B.** Cheap models passed; one expensive model failed outright.

---

5. The year problem — why synthetic years must never reach output

This section exists because the harness is **instructed to write a resume**, and Test B needs years in its input to check whether the model copied them. That creates a real risk worth naming explicitly.

5.1 The instruction creates the exposure

Test B's prompt says, in substance: *copy the education years and certification dates exactly from the source below.* Any model doing its job will therefore emit the years it is given. If the years it is given are synthetic, the output contains fabricated credentials that read as verified.

A **missing** graduation year is incomplete. An **invented** graduation year is a misrepresentation. Those are not the same defect and must not be treated as one.

5.2 Four reasons the rule is absolute

1. **It is a claim to an employer.** A resume is a document of record. A fabricated graduation year or certification date is a false statement attributed to the candidate, in the candidate's name, sent to a third party. No benchmark convenience justifies that.
2. **It is identity-adjacent.** Inventing a year is indistinguishable in kind from inventing the degree. Age and tenure are inferred from graduation years; a wrong year can misstate both.
3. **Nothing downstream would catch it.** Test A's critical gates G1–G4 check employers, degrees and clearance. There is **no gate for years**. A synthetic year that leaked into a production prompt would pass every existing check silently. This is the single most important reason the rule cannot rely on a reviewer noticing.

4. **It cannot be traced back.** These values exist only inside a scorer fixture. No source document supports them, so a leak is not auditable after the fact — there is nothing to verify against.

5.3 What the code guarantees

guarantee	where
production profile contains no education or certification years	`task.CANDIDATE_PROFILE`
fixture block is marked SYNTHETIC in source	`drafter_faithfulness.FIXTURE_EDUCATION`
invariant test fails if the profile gains a year	`test_scorers.py::_invariants`
production path never imports the fixture for real users	`task.build_prompt()` uses `CANDIDATE_PROFILE` only

5.4 The rule, stated plainly

> **Education years and certification dates must always come from the candidate's own uploaded documents.** They are never inferred, never defaulted, never borrowed from a test fixture, and never carried over from another candidate's record. If a source year is unavailable, the resume omits it — omission is always preferable to invention.
>
> If a future change makes the harness copy `FIXTURE_EDUCATION` or `FIXTURE_CERTIFICATIONS` into a prompt for real users, that is a **defect**, not a configuration choice.

---

6. AA-PersonalityBench — a prior, not a score

Data exists for 40 models: 30 trait Elos plus 8 index percentiles, extracted from the website payload because the documented API returns **404 DEPLOYMENT_NOT_FOUND** and the v2 host carries no personality fields.

It is **not** a grading axis, for three measured reasons:

1. **Not exposed by the API** — website payload is the only route.
2. **Does not predict the task** — see the correlation table.
3. **It would mis-rank the winner** — DeepSeek V4.1 Flash has the highest `organization` index of all 40 (100) and finished last among contenders. `eloquent` ranks Claude Fable 5.1 first (1716) and Kimi K3.1 Pro 14th (1189), while the harness found Kimi the best-faithfulness drafter.

Trait vs measured score — n=24 models

One point per model (its best variant). |r| must exceed **0.43** for p<0.05 at n=24.

trait	Pearson r	significant
`scannable`	+0.38	no
`truthful`	-0.29	no
`eloquent`	+0.12	no
`lexicallyVaried`	+0.11	no
`structured`	+0.10	no
`literate`	-0.06	no

No trait clears the threshold. An earlier analysis reported `scannable` at +0.81 on n=6 and called it borderline; at n=24 the same relationship is far weaker. That regression toward zero is

what a small-sample correlation does, and it is why the n=6 figure should never have been quoted as predictive.

Where the traits ARE useful is corroboration: MiniMax M3.1 Flash scores near-bottom `truthful` (202 of a −611…2178 scale) while its `scannable` (1079) exceeds Kimi's (980) — exactly the shape the full-resume test measured independently.

---

7. Corrections carried forward

Recorded because each could have produced a confidently wrong recommendation.

#	error	consequence if unfixed
1	Claimed "AA publishes no writing benchmark"	the benchmark exists on the website; the claim was false
2	Named `slop` as "avoidance", higher-is-better	**inverted** — corr(organization, slop) = −0.196; would have recommended the sloppiest models
3	Parsed personality block `[0]` on every page	a degenerate placeholder produced ~200 identical meaningless rows
4	Predecessor scored 13/27 runs ≥90 while failing a gate	gate clamp added
5	Predecessor weights summed to 110 then clipped at 100	silently compressed every score
6	`rag` substring-matched inside `storage`	flagged a faithful response as fabricated
7	`quantification_score: 0.33` counted as a metric	JSON metadata read as a fabricated number
8	**`2016, 1000, 4.0` in VERIFIED_METRICS but absent from the profile**	a model could invent that graduation year and be *credited* for it by the metrics score
9	Plain-text resumes scored by the JSON scorer	all 43 variants appeared to fail faithfulness
10	`04/2024` parsed as the numbers 4 and 2024	correct dates flagged as fabricated metrics
11	Unmarked bullets not extracted	models with 7 good bullets scored 0
12	Education/cert lines counted as bullets	band compliance forced to 0%
13	Character window instead of line	adjacent credentials shared years, reported as invented
14	Fixed above-then-below line preference	below-layout certs borrowed the previous cert's date
15	Test B-bullets ranked on format	MiniMax M3.1 Flash looked best while inventing 24 items

Errors 8–14 were found by `test_scorers.py` or by inspecting a specific response, not by reading the code. That is the argument for asserting that a scorer fires on known-bad input.

---

8. Limitations

1. **Small n.** Test A is n=4–11 per variant and resolves ~5–6 point differences, not 2-point. Test B is n=3; a 3/3 record is consistent with a ~15% failure rate. Read "no failures observed", not "never fails".
2. **One task each.** One job description, one profile, one fixture. Resume writing is stylistically narrow; this may not transfer to another domain.
3. **Deterministic scorers fail in fixed ways.** Fifteen such errors are listed in §7. The suite in `test_scorers.py` covers the classes found so far; it cannot prove the absence of others.
4. **The Test B prompt is a reconstruction.** The production `stages/drafter.py` prompt has a golden-bullet library and seniority verb tables that vary per deployment. Every truth constraint and the evidence boundary are preserved, so relative comparisons hold; absolute scores are not production-identical.
5. **The fixture is sparse on purpose.** Sparse input is what makes fabrication tempting; it is also not representative of a rich profile.
6. **No LLM-as-judge anywhere.** A judge would inject a second model's bias into a cross-model comparison, so every metric here is deterministic and repeatable.
7. **Cost attribution per key is unavailable.** `/api/v1/activity` has no `api_key_name`, so key totals cannot be joined to models.

---

9. Reproducing

```
source venv/bin/activate

# Test A
python resume_writing_v2/run_contest.py          # round 1
python resume_writing_v2/round2.py               # round 2
python resume_writing_v2/round3.py               # round 3, named set

# Test B (full resume)
python resume_writing_v2/round_b_fullresume.py
RESCORE=1 python resume_writing_v2/round_b_fullresume.py   # no API calls

# verification (free, no network)
python resume_writing_v2/test_scorers.py

# documents
python resume_writing_v2/compile_test_b.py
python resume_writing_v2/compile_results.py
python resume_writing_v2/build_report.py
```

Spend for this document

experiment	USD
Test A round 1 (n=3, all ≤$2/M models)	$0.17
Test A round 2 (n=8, contenders)	$0.20
Test A round 3 (n=4, named set + efforts)	$1.96
Test B-bullets (superseded)	$0.19
Test B full resume	$0.50
total	**$3.03**

Generated from source by `resume_writing_v2/build_report.py`. Prompts, fixtures and allowlists are read from the modules that ran.